

In-Class Exercise/Demo



In-Class Demo

- In today's demo, we will practice middleware on a React app. The middleware we will focus here is redux thunk.
- As you have learned from the lesson slides, redux thunk is nothing more than a function called within and by the function that it's contain in. As you also know, functions can come in different forms – your standard keyword function, arrow function, function expression and method. In a react app, you have seen all the above being used quite often.
- For this demo, we will practice this using codesandbox.
- Please fork this pen immediately when it opens:

<https://codesandbox.io/s/m6-w4-d1-exercise-8qj5s>



In-Class Demo

- Notice some files are already provided on the left panel. The main react app is already established.
- The app uses functional component with `useState` for properties in the main component `Count`.
- In the dependencies, notice *react*, *react-dom* and *react-scripts* are already installed.
- Notice the app on the preview. They don't work at this time.
- We will be implementing `redux` for state management. And then we will use `redux thunk` middleware for `redux` actions to perform data changes in the store.



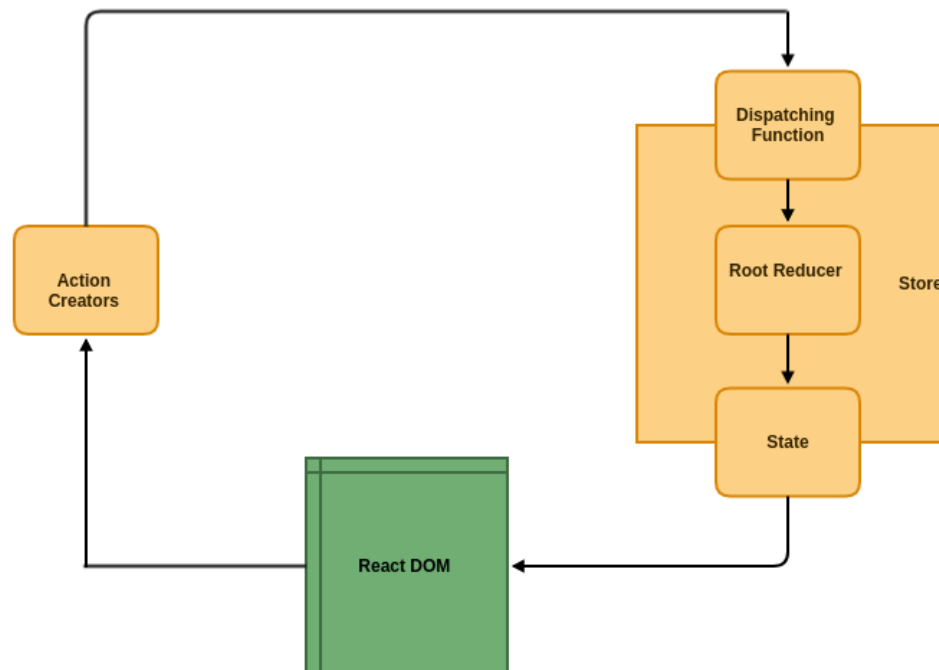
In-Class Demo

- Our first task is to install *redux*, *react-redux* and *redux-thunk*. Add these in the dependencies.
- Next, let's set up our action types and action creators. Inside the actions folder, rename *actionTypes.txt* to **actionTypes.js**. Notice at the top, the actions are organized as `actionTypes`. And below that, are the action creators that is called upon user request. Notice a function is returned by the function it's contain within. We will implement thunk middleware later to help dispatch the actions and payload to the store.
- Next, let's set up the reducer. Inside the reducers folder, rename *counterReducer.txt* to **counterReducer.js**. The reducer sets up the initial state and will also handle the action type received from the actions and perform updates or changes to the state in the store.



In-Class Demo

- Here's a look at an overview of the process:





In-Class Demo

- The next task is to create the store. Although the common practice is to do this using the `mapDispatchToProps()` function, we will opt to do this on a separate file. In the `src` folder, rename `store.txt` to **`store.js`**. Notice in this file, we start by importing the required libraries. We use the `...middleware` spread syntax to allow each middleware to receive store's `dispatch` and `getState` functions as named arguments and returns a function.
- Redux thunk allows you to call your action creators, and instead of returning an action object, it will return another function instead. essentially it will look at all the actions being passed through, and if it's a function it will simply call that function. It also can be passed two arguments, `dispatch` and `getState`. `Dispatch` allows us to dispatch new actions if needed, and `getState` allows us to access the current state.
- Finally, we create the store by calling the `createStore` function and pass the arguments to build our store.



In-Class Demo

- Open App.js which is the main component. First, uncomment/activate the two imports.
- Then in the return JSX elements, add the following:

```
<p>Current Count: {props.count}</p>
<button onClick={props.increment}>+</button>
<button onClick={props.decrement}>-</button>
<button onClick={props.reset}>Reset</button>
```

- The buttons call the respective action creators we'd set up earlier.
- Above the return(), uncomment/ activate the following within the handleSubmit() handler to pass the data as argument to the calling of its action creator:

```
const handleSubmit = () => {
  props.changeCountTo(changeCount);
  setChangeCount(0);
};
```



In-Class Demo

- Next, uncomment/activate the `mapStateToProps()` function. This function sends data from the state in the store to components to be used as props.
- Also, uncomment/activate the export of the redux connection function and main component. This function passes the `mapStateToProps` function and the button call arguments.
- You can now either comment/deactivate or remove the previous export below this ie:

```
//export default Count;
```




In-Class Demo

- The final task is the *index.js* file. This is the gateway to the view that outputs the main component while making sure the entire app (ie. through the main component) have access to the states in the store.
- To do this, will need the react redux's Provider. This element will wrap around the main component while passing the store component to it.
- First, uncomment/activate the two imports: provider and store. Then wrap the main component with the provider element with the store pass as argument:

```
ReactDOM.render(  
  <Provider store={store}>  
    <Count />  
  </Provider>,  
  rootElement  
)
```



In-Class Exercise Demo

DUE:

Today 10.30PM PT

Submission:

Post your codesandbox link on GAP Week 8 Day 1 Exercise



Connect with Us (WEB503/803)

Professor

Rich Loke

richloke@westcliff.edu

Teaching
Assistant

Christopher Paul

christopherpaul@westcliff.edu

Teaching
Assistant

Ilya Valasov

ilyavalasov@westcliff.edu

Teaching
Assistant

Elizabeth Kipp

elizabethkipp@westcliff.edu

End of Presentation